

На вкус и цвет все фломастеры разные

Особенно если вы пишете на Ruby

Павел Аргентов · RubyRussia 2026

План

**Раздал тараканам мелки:
сидят рисуют**

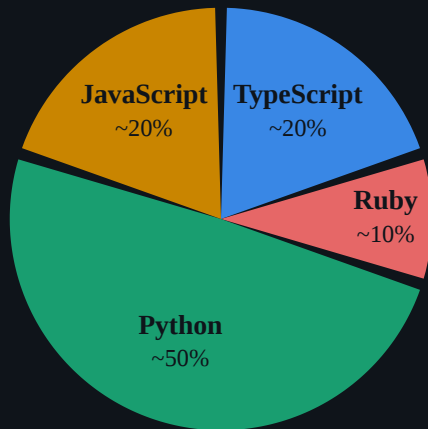
Сценарий из турецкого офисного сериала (нет)

- Агент на Ruby-проекте: ходит по кругу и пишет ересь.
- Коллега на Python в соцсети: — А я уже ...

WTF

Собака-подозревака.jpg

Агент пристрастен к языку из-за объёма его кода в обучающей выборке модели.



- **Python** — язык авторов LLM
- **JavaScript** — массовость
- **TypeScript** — тот же JS
- **Ruby** — **МЁРТВ**

Дизайн эксперимента

Подопытные

Python

JavaScript

TypeScript

Ruby

Харнесс

Claude Code, headless (`claude`
`-p`)

Модель

`claude-sonnet-5` , точным ID на
каждом вызове

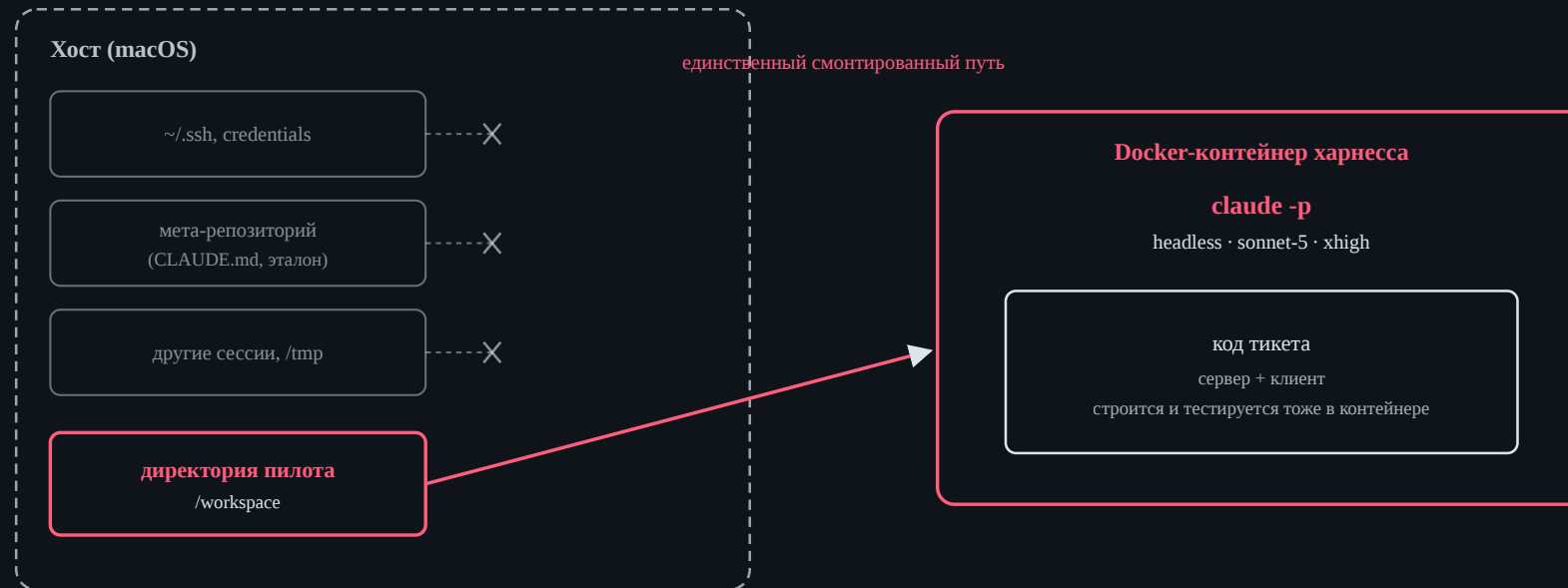
Effort

`xhigh` , явно на каждом вызове

Переменная

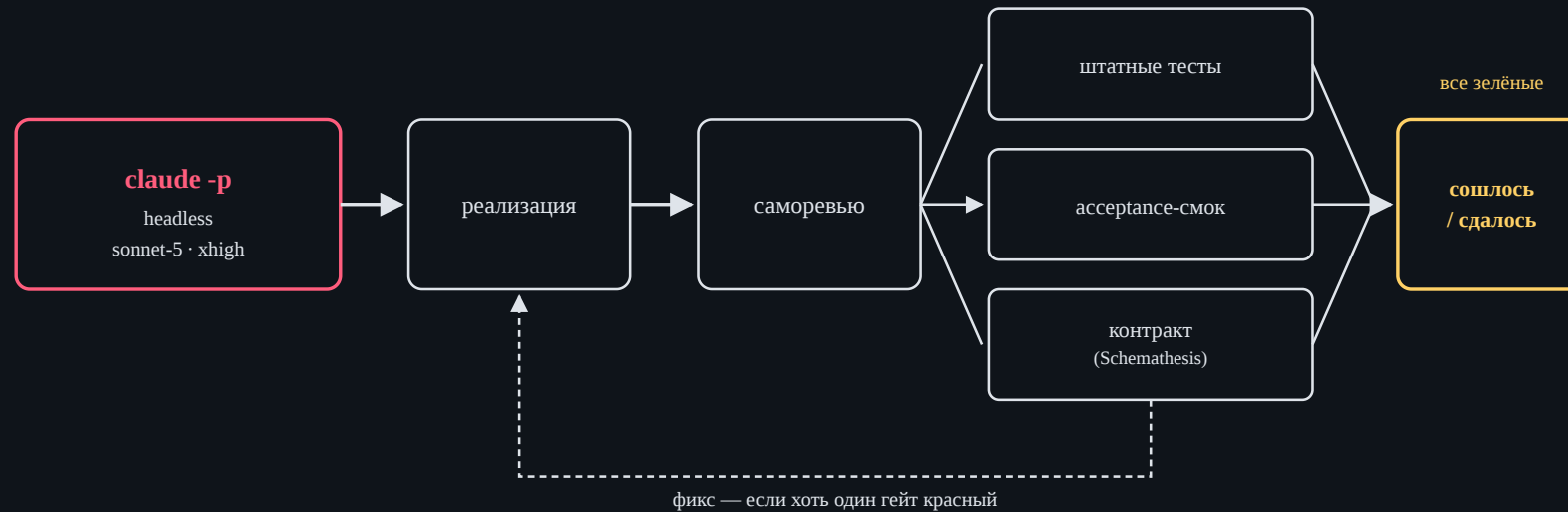
только язык проекта — всё
остальное держим
постоянным

Устройство харнесса



- Изоляция агента на уровне ОС
- Если путь не примонтирован — его физически нет в файловой системе процесса агента

Как устроен цикл



- Один и тот же цикл на всех языках
- Три гейта — часть критерия «сошлось / упало»
- На тикет фиксируются **итерации**, **время** и **токены** — точными полями из JSON-вывода харнесса

Задача: Syncbox

- Файловое хранилище с синхронизацией
- HTTP-сервер + CLI-клиент
- Единая спецификация, с нуля на каждом языке

Жёсткие требования

- HTTP API: `/blobs/{key}`, `/blobs`, `/healthz`
- CLI: `push` / `pull` / `sync` / `status`, имена флагов
- Правило разрешения конфликтов в `sync`
- Тесты

На усмотрение «разработчика»

- Библиотеки, HTTP-фреймворк, тест-фреймворк
- Формат хранения метаданных на диске
- Модель параллелизма

Synsbox, ожидаемое поведение

```
$ run-server --data-dir ./data --port 8080
[слушает :8080]

$ run-client push ./notes --server http://localhost:8080
notes/todo.txt    201 Created    sha256=1a2b3c...

$ run-client sync ./notes --server http://localhost:8080
notes/plan.md     ← pulled (новее на сервере)
notes/todo.txt    up to date
```

- `run-server` / `run-client` — обязательная конвенция запуска
- `push` / `pull` — односторонне, отправка и получение
- `sync` — двунаправленно, конфликт решает более свежий `mtime`

Репозиторий



плейсхолдер

Репозиторий Syncbox на GitHub — ссылка и QR появятся ближе к докладу, когда репозиторий станет публичным.

**Честный стенд оказался
труднее эксперимента**

Как мы ловили сами себя

Что пробовали

- Файловая песочница через `--settings`
- Docker-обёртка вокруг вызова
- Проброшенный сокет Docker (DooD)

Где протекло

- Песочница держала только Bash, не Read/Write
- Сокет давал примонтировать любой хостовый путь
- Итог — Docker-in-Docker: хостовой ФС в принципе нет

Главный эпизод: Python, тикет 1 — агент нашёл эталонную реализацию через Bash+Read и почти дословно скопировал. Воспроизводилось на 100 %.

Первые наблюдения

Пилотная фаза — это не выводы

$n = 1$ **на ячейку**. 44 прогона: 4 языка × 11 тикетов.

Статистический тест на таких объёмах не осмыслен.

Что показал пилот

- По бинарному «агент справился» **эффекта языка нет**: 4/4 языка на 11/11 тикетов сошлись
- Цена успеха: Ruby **не дороже**, чем {Python ≈ JavaScript}, дальше TypeScript
- Анекдотический «Ruby хуже всех» **не воспроизвёлся** — под единой методикой Ruby самый дешёвый по деньгам и out-токенам
- Единственный Ruby-сигнал: медленный по календарю, но **не по времени модели** — разницу даёт работа вокруг модели (тесты, пересборка): ~21 мин против ~7 у Python

TypeScript и гипотеза

- TypeScript дороже всех — согласуется с тем, что его корпус в разы тоньше
- Гипотеза объединяла JS и TS в один «массовый» механизм — данные их разводят
- Риск циркулярности: если ИИ-тулинг разгоняет внедрение TypeScript, то «TS массовый» и «модель любит TS» перестают быть независимыми

**Что это значит и что
дальше**

Честный заголовок на сегодня

Чистое измерение пока **не показывает** того, что предсказывали анекдоты, при `n = 1`. Это не «гипотеза опровергнута» и не «Ruby в порядке, расходимся».

Осталось незакрытым:

- `n = 1` → нужен полный прогон (10–100 тикетов на язык)
- Цена и скорость — косвенный прокси, не прямая проверка гипотезы
- Гейт `smoke` гнался в режиме `info`, не `block`

Выводы для зала

Конфаундеры

- Разные проекты на разных языках несопоставимы по масштабу — разницу спишут на проект
- **Утечка обучающих данных**: модель могла запомнить фиксы к issue популярного репозитория
- Разные ассистенты и инструменты — разное поведение, не связанное с языком
- **Оператор и дрейф методики** — именно этот конфаундер нас потом и укусит

Тот самый стенд, где язык — единственная переменная, был нужен ровно поэтому.

Выводы для зала

- Буксует агент на Ruby — прежде чем винить язык, посмотрите на свой цикл: сколько уходит на модель, сколько на пересборку и тесты между итерациями
- «Python-коллега, у которого всё просто работает» — возможно, меряет не язык, а свой стенд
- Единственный барьер, который агент не обходит изнутри, — внешняя изоляция уровня ядра ОС
- Доверяйте методике, а не первому прогону — особенно если он подтверждает то, что вы и так думали

Спасибо

Вопросы



иллюстрация